

Glossary

Acknowledgment Device response indicating successful reception of message

Activation record Temporary storage in memory for function's preserved registers, arguments, local variables, return address, etc. Exists only from function's start to end.

Aliasing Distortion of signal resulting from sampling at too low of a frequency

Ammeter Test device which can measure current value through circuit. Multimeters typically have ammeter modes available.

Analog Able to represent an infinite number of possible values

Analog-to-digital converter (ADC) Circuit which converts an analog value (e.g. voltage) to its corresponding digital value

And Multi-input binary logic operation with output of one only if all inputs are one, else output is zero

Anode Positive terminal of a polarized component (LED, battery, etc.)

Arithmetic/logic unit (ALU) Hardware circuit in CPU which performs a machine instruction's arithmetic and logic operations

Assembler Software tool which translates assembly language code into machine code

Assembly language Human-readable representation of machine code

Asynchronous Activities which are not synchronized with each other, or a protocol that does not send clocking information

Atomic Indivisible, cannot be interrupted or preempted

Baud rate Rate at which communication symbols are transmitted. Also called symbol rate.

Big-endian Describes byte ordering convention in which the most significant byte is stored first in memory

Binary Base-two numbering system. Each digit can have one of two values (zero and one).

Bitwise and Operation in which output bit is logic *and* of corresponding input bits. C operators are `&` and `&=`.

Bitwise one's complement Operation in which output bit is inverse (one's complement) of corresponding input bit. C operators are `~` and `~=`.

Bitwise or Operation in which output bit is logic *or* of corresponding input bits. C operators are `|` and `|=`.

Blocking State in which a task is waiting for an event to occur. Also called waiting.

Burst-mode Mode in which DMA controller performs all transfers in a burst without sharing bus with CPU

Busy-waiting Wasteful method of making a program wait for an event or delay. Program executes test code repeatedly in a tight loop, not sharing time with other parts of program.

Byte Value which is eight bits long

Byte-addressable Memory in which each address identifies a single byte

Call graph Diagram showing subroutine calling relationships between functions in a program

Call stack Stack of activation records/stack frames of functions which have started executing but have not yet completed

Cathode Negative terminal of a polarized component (LED, battery, etc.)

Central processing unit (CPU) Hardware circuit that executes a program's instructions

Clear To change a bit to zero

Clock gating Method to disable circuit by blocking clock signal, reducing power consumption

CMSIS-CORE Portion of CMSIS that provides C-language interface to processor core and peripherals

Comparator Circuit that compares two values to determine equality or identify larger value

Compiler Software tool that translates high-level source code to assembly language code

Condition code flag Indicates whether result of instruction is negative (N) or zero (Z), or whether instruction resulted in carry (C) or overflow (V)

Control register Register used to configure operation of hardware in CPU or peripheral

Cooperative multitasking Scheduling approach where tasks share CPU by voluntarily yielding it to other tasks

Cortex Microcontroller Software Interface Standard (CMSIS) Definition of hardware/software interfaces and debugging interfaces that simplify the development of systems with Cortex-M processors

Counter Digital circuit which counts number of input pulses

CPU overhead Portion of time CPU spends executing code that does not perform useful work for the application

Critical section Section of code that may execute incorrectly if not executed atomically

Cycle-stealing Mode in which DMA controller shares bus with CPU, taking turns to transfer data

Data race Situation where ill-timed preemption of a code critical section can result in incorrect program result

Decimal Base-ten numbering system. Each digit can have one of ten values (0 through 9).

Demultiplexer Electronic selector switch that routes input signal to one of N outputs

Deserialization Conversion of information from serial to parallel form

Digital Capable of taking on a limited number of values

Digital-to-analog converter (DAC) Circuit that converts a digital value to its corresponding analog value (e.g. voltage)

Direct memory access (DMA) Type of memory access performed by peripheral hardware without program instructions

Directive In assembly language, an order to control how assembler operates. Does not represent an instruction.

DMA controller Peripheral that performs DMA (Direct Memory Access)

DMAMUX Multiplexer that selects DMA event source

Do not populate (DNP) Indicates that a PCB component is optional and is not installed

Duty cycle Fraction of time that a PWM signal is asserted

Endianness Property that describes the order of bytes in multi-byte structures stored in memory

Energy (W) Capability of a system to do work on another system. Measured in Joules (J). Symbol is W (work).

Epilog Final code in function which restores preserved registers, prepares return values, frees activation record and returns control to caller function

Event-triggering Approach in which software runs when an event occurs

Exception Event that causes a program to deviate from normal flow of control. Examples include illegal instruction, illegal memory access, and interrupt.

Field A group of one or more bits defining a data item. A register may hold multiple fields.

Finite state machine (FSM) A type of state machine with all states and transitions defined

Framing symbol Symbol used to indicate start or end of message

General-purpose input/output port (GPIO port) Peripheral with digital input and output bits

General-purpose register Register located in CPU used for data processing by instructions in program

Halfword Value that is 16 bits (two bytes) long

Handler Software routine that runs in response to interrupt or exception request

Hexadecimal Base-sixteen numbering system. Each digit can have one of sixteen values (0 through 9, A, B, C, D, E and F). Symbols A through F represent values of ten through fifteen.

Immediate value Data value that is stored as part of a machine instruction

Infrared (IR) Electromagnetic energy immediately past the visible portion of the spectrum; also called invisible light

Input GPIO port bit Portion of GPIO port that enables program to read a single-bit input signal

Instruction Command for processor to execute. Consists of an operation and zero or more operands.

Instruction set architecture (ISA) Description of instructions, registers, and memory accessing modes that a CPU supports

Integrated circuit (IC) Electronic circuit with components built into a single piece of silicon, enabling extreme miniaturization, mass production, and cost reduction

Integrated development environment (IDE) PC-based program supporting development activities such as code editing, building, downloading, debugging

Inter-Integrated circuit bus (I²C) A type of synchronous serial communication bus with addressing and acknowledgments

Interrupt Event used to trigger specific program activity

Interrupt request (IRQ) Hardware signal indicating that an interrupt is requested

Interrupt service routine (ISR) Software routine that runs in response to interrupt request. Also called a handler.

Invert To change a bit to the opposite value. Also called toggle.

Kernel Scheduler with support for task features such as communication, delays, and synchronization

Label Symbol in assembly language which represents an address

Least-significant Having the smallest place value. The least-significant byte of a two-byte value represents values of 0–255.

Light-emitting diode (LED) Electronic component which emits light. Used for indicators, backlighting, and general illumination.

Link register (LR) ARM CPU register that holds return address for subroutine calls or return code for exception handlers

Linker/Loader Software tool that combines separate object code modules and links cross-references to create single executable program file

Little-endian Describes byte ordering convention in which least-significant byte is stored first in memory

Local variable Variable that is visible and accessible only within its declaring function

Machine language Code in which each instruction is represented as a numerical value. Processed directly by CPU.

Media access control (MAC) Rules controlling when a node can transmit a message on shared media

Microcontroller unit (MCU) Integrated circuit containing CPU, peripherals, support circuits, and often memory

Mnemonic In assembly language, text abbreviation used to describe operation performed by instruction

Modularity Measure of how program is structured to group related portions and separate independent portions

Most-significant Having the greatest place value. The most-significant byte of a two-byte value represents values of 0 to 65,280 which are multiples of 256.

Multimeter Multi-function test equipment which can measure electrical values such as voltage, current, and resistance

Multiplexer Electronic selector switch that routes one of N inputs signals to the output. MCU pin multiplexer is bidirectional (includes demultiplexer).

Multitasking Approach in which program consists of multiple tasks with independent control flow interleaved over time

Native data type Primary data type used by ALU and registers; 32-bit integer for ARM Cortex-M CPUs

Non-preemptive scheduler Scheduler that does not allow tasks to preempt each other

Operand Part of an instruction: parameter used by operation

Operating system Kernel with support for application-oriented features such as file systems, networking support, etc.

Operation Part of an instruction: specifies what work to do

Or Multi-input binary operation with output of one if any inputs are one, otherwise output is zero

Output GPIO port bit Portion of GPIO port that enables program to write a single-bit output signal

Parallel Organization in which multiple items are simultaneously available or active

Pending Requested but not yet serviced (e.g. interrupt)

Peripheral Hardware that helps CPU by interfacing or providing special functionality

Polling Software approach in which program explicitly checks a condition

Pop Instruction which reads a data item from the top of the stack (last used location) in memory and updates the stack pointer

Power (P) Rate at which a device uses energy. Measured in Watts (W). Symbol is P.

Preemption Pausing the execution of a task to allow another task to run

Preemptive scheduler Scheduler which supports task preemption

Printed circuit board (PCB) Board which holds electronic components and conductive traces for interconnection

Prioritization Favoring one item over another. For example, running task A before B to reduce A's latency at the expense of B.

Program counter (PC) CPU register used to specify address of instruction to execute next

Program status register (PSR) Register holding condition code flags

Programmer's model Specifies a CPU's characteristics, including instructions, data types, registers, addressing modes, and operating modes

Prolog Initial code in function which preserves registers and prepares activation record

Pulse-width modulation (PWM) Method for encoding information onto a single digital signal based on duty cycle

Push Instruction that writes a data item to the next free stack location in memory and updates the stack pointer

Quantization Process of selecting a discrete digital value to represent an analog value

Real-time kernel Kernel designed for real-time systems

Real-time operating system (RTOS) Operating system designed for real-time systems

Real-time system System that must respond to events before specified deadlines

Register Hardware circuit which can store a data value

Register file Holds CPU's general purpose registers

Responsiveness Measure of how quickly a system responds to an input event

Return address Address of next instruction to execute after completing a subroutine

Root function A task's main software function, which may call other functions as subroutines.

Sampling Process of converting a continuous-time signal to a series of discrete-time samples

Scheduler Mechanism to control which task runs on a processor at a given time

Sequence diagram Diagram showing sequence of operations and communications between two or more actors (e.g. threads, peripherals)

Serial Organization in which items are available or active sequentially, not simultaneously

Serial peripheral interconnect (SPI) A type of synchronous serial communication bus

Serialization Conversion of information from parallel to serial form

Set To change a bit to one

Signed Numbering system that is able to represent positive and negative values and zero

Spaghetti code Code which is poorly structured because it entangles unrelated features, complicating development and maintenance.

Stack Last-in, first-out data structure. Data items are removed (popped) in the opposite order they were inserted (pushed).

Stack pointer (SP) Pointer to data item on top of stack (last in, first out)

State machine State-based model of system with rules for transitions between states

Status register Register that indicates the status of hardware in CPU or peripheral

Subroutine Program function that can be called by another function

Supply voltage Level of voltage applied to electronic circuit to enable operation. Also called VDD or VCC.

Symbol (communication) A waveform or state transmitted on a communication channel to represent one or more bits of information.

Symbol (program) Text name representing a value (e.g. address, data value) in a program

Synchronous Activities which are synchronized with each other, or a protocol which sends clocking information

SysTick timer Timer peripheral available in Cortex-M CPU cores, typically used to generate periodic time tick

Task Function and its subroutines that perform an activity. Each task has its own flow of control.

Task preemption Scheduling approach where a task is paused to allow a different task to run. Eventually the first task resumes execution where it was paused.

Timer/counter Peripheral that measures time or counts events

Timer/PWM module (TPM) Timer peripheral in Kinetis KL25Z MCU which can also generate PWM signals

Top-of-stack Next item that can be popped from stack

Transfer function Mathematical equation describing relationship between input and output values

Transistor Basic electronic component which operates as switch or amplifier

Universal asynchronous receiver/transmitter (UART) Peripheral for asynchronous communications

Unsigned Numbering system that is able to represent positive values and zero

Vector Address of an exception handler

Vector table Table of vectors used to process different exceptions

Volatile data Data that can change outside of program's normal flow of control

Volatile memory Memory that loses its contents if power is lost

Voltmeter Test device which can measure voltage value across circuit. Multimeters typically have voltmeter modes available.

Waiting A state in which a task is waiting for an event to occur. Also called blocking.

Watchdog timer (WDT) Hardware peripheral used to reset out-of-control program

Word Value that is 32 bits (four bytes) long

Index

- Accessing data in memory
 - array elements, 159–62, 159f, 161f
 - automatically allocated memory, 158
 - dynamically allocated memory and other pointers, 159
 - statically allocated memory, 157–58
- Acknowledgments, 241
- Activation record, 141
- Addressing, 242, 271–72
- Aliasing, 171
- Analog, 13, 168
- Analog comparator
 - concepts, 176, 177f
 - voltage transition monitor, example application, 181–83
- Analog interfacing, 21
 - concepts, 168, 169f
 - quantization, 168, 169–71, 169f, 170f
 - sampling, 168, 171–72
- Analog signals, 13
- Analog waveform generator. *See* waveform generator
- Analog-to-digital conversion (ADC), 13, 168
 - concepts, 184
 - converter architectures, 184–85, 185f
 - example applications
 - hotplate temperature sensor, 192–95, 193f
 - infrared proximity sensor, 195–201, 195–98f
 - inputs, 185, 185f
 - triggering, 186
- And operation, 37, 102
- Anemometer, 208, 208f, 225–29
- Anode, 31
- Application program status register (APSR), 96
- Architecture, 94–95
 - instructions, 100–106
 - memory, 96–100
 - operating behaviors, 107
 - registers, 95–96
 - Thumb instructions, 106–107
- Arithmetic/logic unit (ALU), 93
- ARM Cortex-M0+ processor, 17–19, 18f, 19f. *See also* CPU; processor
- ARMv6 architecture profile, 95. *See also* architecture
- ARMv6-M architecture profile, 95. *See also* architecture
- Array elements, 159–62, 159–61f
 - one-dimensional arrays, 161
 - two-dimensional arrays, 162
- Assembler, 100, 134, 136–38
- Assembly language, 100, 137–38
- Asynchronous, 108, 240
 - Asynchronous exceptions and interrupts, 108
 - Asynchronous serial communications
 - communicating with a PC (example), 260
 - interrupt-driven communications, 265–69
 - polled communications, 264–65
 - protocol concepts, 249
 - Atomic object access, 128–30
 - Automatically allocated memory, 158
- Baud rate, 239, 253, 256, 260, 274
- Big-endian systems, 97, 98f. *See also* Little-endian systems
- Binary, 34
- Bitwise and, 102, 247
- Bitwise one's complement, 102
- Bitwise or, 34
- Blocking. *See* waiting
- Burst-mode, 286
- Busy-waiting, 64, 78
- Byte, 96–98
 - least-significant, 97
 - most-significant, 97, 98
- Byte-addressable, 96
- C control flow structures
 - calling subroutines, 156–57
 - conditionals, 148–52
 - if/else statements, 148–49, 149f
 - switch statements, 150–52, 151f
 - loops
 - do while code, 152–53, 153f
 - for code, 155–56, 155f
 - while code, 153–54, 154f
- C language fundamentals
 - memory types, 141–42
 - program and functions, 140–41
 - program's memory requirements, 142–43
 - start-up code, 141
- Call graph, 140
- Call stack, 141
- Cathode, 31
- Central processing unit (CPU), 6, 7, 13, 14, 28. *See also* CPU
 - core; CPU overhead
 - exceptions and interrupts. *See* exceptions and interrupts
 - and GPIO peripheral, 43–45, 44
- Clear, 37, 45
- Clock gating, 38–39
- Clock signal, 20, 38
- CMSIS-CORE, 34, 36, 36f, 41

- Communication interfaces, 20–21
- Comparator, 176, 177–81, 177–78*f*, 180*f*, 179*t*, 180*t*
- Compiler, 128
- Concurrency, 13–14
 - advanced scheduling, 82
 - real-time systems, 85–86
 - task preemption, 84–85, 85*f*
 - task prioritization, 83–84
 - waiting, 82–83
- concepts of, 61
 - creating and using tasks, 65–66, 67*f*
 - starter program, 61–65
- improving responsiveness, 69
 - interrupts and event triggering, 69–74
 - reducing task completion times with finite state machines, 74–77
 - using hardware to save CPU time, 77–81
- overview, 60
- Condition code flags, 96, 101
- Control flow instructions, 104–105. *See also* C control flow structures
 - conditional branch, 104–106
 - subroutine, 105–106
- Control register, 32, 96, 173
 - and C Code, 34
 - and CMSIS, 34
 - coding style for accessing bits, 34–36
 - reading, modifying, and writing fields in, 36–38
- Cooperative multitasking, 67
- Cortex Microcontroller Software Interface Standard (CMSIS), 34
- Counters, 78, 206, 208. *See also* Timers
- CPSID and CPSIE instructions, 106
- CPU. *See* Central processing unit (CPU)
- CPU core
 - architecture, 94–95
 - instructions, 100–101
 - memory, 96–100
 - operating behaviors, 107
 - registers, 95–96
 - Thumb instructions, 106–107
 - concepts, 92–94
 - simplified structure of, 93*f*
- CPU overhead, 60
- Critical section, 121–22
- Curve-fitting, 192
- Cycle-stealing mode, 286

- Data movement instructions, 101–102
- Data processing instructions, 102–105
- Data race, 129
- Debug microcontroller, 21
- Debugger, 135, 139*f*
- Decimal, 34
- Deserialization, 239
- Development board, 21–22, 22*f*
- Diagnostics, 16, 16*f*
- Digital, 13, 30–31
- Digital-to-analog converter (DAC), 13, 168
 - concepts, 172–73
 - control register, 173, 174*f*
 - converter architectures, 173
 - waveform generator, example application, 174–76, 176*f*
- Direct memory access (DMA), 174
 - concepts, 286
- Direct memory access multiplier (DMAMUX)
- Directive, 137
- Duty cycle, 209
- Dynamic random access memory (DRAM), 142
- Dynamically allocated memory, 159

- Electric hot plate, 4*f*
 - internal components, 4*f*
 - temperature control system, 5–6, 5*f*
 - block diagram of a computer-controlled hot plate, 8*f*
 - and electronics, 6
 - and embedded computer, 6–7
- Electrically erasable programmable ROM (EEPROM), 142
- Electronics, in temperature control system, 6
- Embedded system/computer, 23
- Embedded system/computer (generally), 6–7
 - attributes
 - concurrency, 13–14
 - constraints, 16–17
 - diagnostics, 16, 16*f*
 - interfacing with inputs and outputs, 13
 - reliability and fault handling, 15
 - responsiveness, 14–15
 - embedding method, 7–8
 - energy constraints, 17
 - microcontrollers, 6
 - parts costs, 17
 - power constraints, 17
 - size constraints, 17
 - software operations
 - closed-loop control, 12
 - communications and networking, 12
 - sequencing, 12
 - signal conditioning and processing, 12
 - temperature constraints, 17
 - weight constraints, 17
- Endianness, memory, 97
- Epilog code for functions, 143, 145–46, 148
- Error detection, 241
- Event-triggered approach, 70
- Exception handlers, 147–48
- Exceptions and interrupts, 107–108
 - exception handling behavior
 - entering a handler, 109
 - exiting a handler, 110, 110*f*
 - Handler mode and stack pointers, 107
 - hardware
 - Cortex-M0+ exception sources, 112, 112*t*
 - exception making, 111
 - exception sources, vectors, and handlers, 111, 112*t*
 - NVIC operation and configuration, 119
 - overview, 110*f*, 111
 - peripheral interrupt configuration, 114

- vector table definition and handler names, 113–14
- software for interrupts
 - interrupt configuration, 125–26
 - program design, 112
 - sharing data safely given preemption, 127–30
 - writing ISRS in C, 126–27
- Execution program status register (EPSR), 96
- External references, C code, 136
- Fault handling, 15
- Field, 34
- Finite state machine (FSM), 74
 - analysis, 75–77
 - program structure, 75–77
- Flash memory, 20
- Flash ROM, 142, 143f
- Framing symbol, 240
- Functions
 - activation record creation, 146, 146f
 - activation record deletion, 147, 147f
 - body, 143, 145
 - epilog, 143, 146–47, 148
 - exception handlers, 147–48
 - prolog, 143, 145–46, 148
 - register use conventions, 144
 - function arguments, 144
 - function return value, 145
- General purpose input/output (GPIO)
 - additional pin configuration options
 - high current drive outputs, 54
 - basic digital input and output circuit, 28f
 - inside the MCU, 32–55
 - outside the MCU, 30–32
 - overview, 28
 - peripheral, 43
 - assemble the complete program, 46
 - data in register, 43
 - data out register, 44
 - logic components, 44f
 - module use 44–45, 45f
 - pin configuration options, 52–55
- General-purpose register, 101, 144, 147
- Half-word, 103, 106
- Handlers, 70, 111–14
 - exception handler
 - reset, 141
 - exception handlers, 147–48
 - exception handling behavior
 - entering a handler, 109
 - existing a handler, 110, 110f
 - Handler mode and stack pointers, 107
 - interrupt handler, 183
 - operating behaviors, Thread and Handler modes, 107
- Hexadecimal, 32, 138, 243
- High-level programming languages, 134
- Hold circuit, 185
- Hotplate temperature sensor, ADC, 192
- If/else statements, 148, 149f
- Immediate value, 101
- Infrared proximity sensor, ADC, 195, 196f
 - circuit description, 196–99, 197
 - control software, 199–201
- Input GPIO port bit, 28, 43
- Input/output path configuration, 38, 38f
 - clock gating, 38–39
 - connecting pin to peripheral module, 39–43
- Instruction set architecture, 94
- Instructions, 6, 93, 100–106, 101t, 137
 - control flow instructions, 104
 - data movement instructions, 101–102
 - data processing instructions, 102–103
 - execution versus debugging, 107
 - memory access instructions, 103–104
 - miscellaneous instructions, 106
 - Thumb instructions, 106–107
- Integrated development environment (IDE), 135
- Interfacing
 - analog interfacing, 21
 - concepts, 168, 169f
 - quantization, 168, 169–71, 169f, 170f
 - sampling, 168, 171–72
 - examples
 - driving a 3-color LED on Freedom board, 47–49
 - driving a speaker, 50–51, 51f
 - driving the hot plate's heating element, 51
 - with inputs and outputs, 13
 - with switches and LED lights, 28f, 31–32
- Inter-integrated circuit bus (I²C)
 - message format, 270–71
 - device addressing, 271–72
 - register addressing, 272
 - protocol concepts, 269–70
- Interrupt handler, 183
- Interrupt program status register (IPSR), 96
- Interrupt request (IRQ), 70
- Interrupt service routine (ISR), 70, 122, 227
 - C code, 131
- Interrupt system, 70. *See also* Exceptions and interrupts
- Interrupt-driven communications, 265–69
- Interrupts and event triggering. *See also* Exceptions and interrupts
 - analysis, 72–74
 - program structure, 70–72, 71f
- Keil μ Vision (microVision) IDE
 - software development. *See* software development tools
- Kernel, 82
- Label, 138
- Least-significant byte, 103
- Light-emitting diodes (LEDs), 8–9, 28, 28f, 29f, 31–32, 44–45, 123
 - dimming, 231–33
- Link register, 96, 104, 106, 145, 146, 147
- Linker/loader, 139
- Linux OS, 17

- Liquid crystal display (LCD), 14
- Little-endian systems, 97, 98*f*. *See also* Big-endian systems
- Load/store architecture, 95
- Local variable, 76
- Machine language, 100
- Media access control (MAC), 239, 241–42
- Memory
 - data access. *See* accessing data in memory
 - endianness, 97
 - memory map, 96–97
 - stack, 98–100
- Memory access instructions
 - load/store instructions, 103–104
 - memory addressing, 103
 - stack instructions, 104
- Microcontroller unit (MCU), 6–7, 14, 28
 - accessing bits, coding style for, 34–36
 - additional pin configuration options, 52
 - high current drive outputs, 54
 - pull-up resistors for inputs, 52–53
 - assembling the complete program, 46
 - GPIO peripheral, 43–45
 - input signals, 30–31
 - interfacing examples
 - driving a-color LED on Freedom board, 47–49
 - driving a speaker, 50–51, 50*f*
 - driving the hot plate's heating element, 51–52
 - interfacing with a switch and LED lights, 28*f*, 31–32
 - I/O path configuration, 38–43
 - output signals, 31
 - using CMSIS to access hardware registers with C Code, 34
- Microcontrollers, 6, 20, 21*f*
 - components within, 92*f*
 - flash memory, 20
 - peripheral devices, 20
 - SRAM, 20
- Mnemonic, 100, 101*t*
- Modularity, 60
- Most-significant byte, 97, 98
- Multiplexer, 39–40, 40*f*, 179*t*, 185–88
- Multitasking, 67, 84
- MUX control bits, 42–43
- Native data types, 95
- N-bit resistor ladder, 173
- Negative temperature coefficient (NTC), 192, 193*f*
- Nested Vectored Interrupt Controller (NVIC), 20
 - operation and configuration
 - enable, 120
 - pending, 120
 - priority, 120
- Non-preemptive scheduler, 73
- NOP instructions, 106
- Object code disassembling tools, 138–40
- Operand, 93
- Operating behaviors
 - instruction execution versus debugging, 107
 - Thread and Handler modes, 107
- Operating system (OS), 14, 17, 82
- Operation, 93
- Optimization, of code, 15
- Or operation, 42
- Output GPIO port bit, 28
- Parallel, 238, 239
- Parameters, 140
- Pending, 120
- Periodic 1 Hz interrupt, 212
- Periodic timer tick, 207
- Peripherals, 14, 15, 20, 21, 28
- Photovoltaic (PV) cell, 17
- Pointers, 159
- Polled communications, 264–65
- Polling, 69–70
- Popping data, 98, 104
- Power constraints, 17
- Preemptive scheduler, 84–85, 127
- PRIMASK register, 96, 106
- Printed circuit board (PCB), 8–12, 28, 39
 - of refrigerator, 9, 11*f*, 12
 - of remote-controlled quadcopter toy, 8–9, 10*f*
- Prioritization, 83–84
- Processor, 15, 18–19, 20*f*. *See also* CPU
 - clock signal, 20, 42
- Program counter (PC), 93
 - communicating with, 262–69
 - interrupt-driven communications, 265–69
 - polled communications, 264–65
- Program status register (PSR), 96
- Program-counter-relative addressing mode, 157
- Programmer, 139
- Programmer's model, 94
- Prolog code for functions, 143, 143, 145–46, 148
- Proximity sensor, ADC, 189*f*, 190*f*, 193*f*, 195*f*, 195–202
- Pull-up resistors for inputs, 52–53
- Pulse-width modulation (PWM) mode, 173, 209
- Pushing data, 98, 104
- Quantization, 168, 169–71
 - transfer function, 171
- Random access memory (RAM), 142, 143*f*
- Read-only memory (ROM), 142
- Real-time kernel (RTK), 86
- Real-time operating systems (RTOS), 14, 86
- Real-time scheduling analysis, 85
- Real-time systems, 85–86
- Refrigerator
 - embedded computer, 8, 9*f*
 - embedded system attributes, 12–17
 - hardware, 9, 11*f*, 12
 - software operation, 12
- Register file, 93, 94
- Registers, 93, 94–97, 95*f*
- Reliability, of embedded system, 15
- Remote-controlled quadcopter toy

- embedded computer, 8, 9*f*
 - embedded system attributes, 12–17
 - hardware, 8–9, 10*f*
 - software operation, 12
- Reset exception handler, 141
- Resistors, 31–32
- Responsiveness, 60, 68–70
 - of embedded system, 14–15
 - raw processing speed, 14–15
 - task scheduling, 15
- interrupts and event triggering, 69–73
- reducing task completion times with finite state machines, 74–78
- using hardware to save CPU time, 77–81
- Return address, 106
- Root function, 65
- R-2R resistor ladder, 173
- Sample circuit, 185
- Sampling, 168, 171–72
 - aliasing, 171
 - signal's spectrum, 171–72, 171–72*f*
- Schedulable system, 85
- Scheduling/scheduler, 14, 15
 - advanced, 82
 - real-time systems, 85–86
 - task preemption, 84–85, 85*f*
 - task prioritization, 83–84
 - waiting, 82–83
 - non-preemptive scheduler, 73
 - preemptive scheduler, 84–85, 127
- Secure Digital flash cards, 14
- Sequence diagram, 79, 80*f*
- Serial communications
 - concepts, 238–49
 - development tools, 242–44
 - methods, 239
 - acknowledgments, 241
 - addressing, 242
 - error detection, 241
 - media access control, 241
 - message framing, 240
 - serialization, 239–40
 - symbol timing, 240
 - protocols and peripherals, 249
 - asynchronous serial communications, 257–69
 - inter-integrated circuit bus (I2C), 269–81
 - synchronous serial communications, 249–57
 - reasons, 238–39
 - software structures, 244
 - queue implementation, 246–49
 - queue use, 249
 - supporting asynchronous communication, 244–45, 245*f*
- Serial peripheral interface (SPI), 239, 249
 - loopback test, 255–57
- Serial wire debug (SWD), 140
- Serialization, 239–40
- Set, 36–37
- Signal
 - frequency of, 208
 - period of, 208
- Signed extend instruction, 102
- SIM_SCGC5 control register, 32, 33*f*, 36–37
- Software development tools, 135
 - debugger, 140, 139*f*
 - program build tools, 135, 135*f*
 - assembler, 136–38
 - compiler, 136
 - linker/loader, 139
 - programmer, 139–40
- Software for interrupts
 - interrupt configuration, 125–26, 124*f*
 - program design
 - communication, 123–24
 - example system design, 124–25
 - partitioning, 123
 - sharing data safely given preemption, 127–30
 - writing ISRs in C, 126–27
- Software–hardware interactions, 79–81, 80*f*
- Solid-state relay (SSR), 52, 52*f*
- Spaghetti code, 65, 74
- SP-relative addressing mode, 158
- Stack, 98–100, 99–100*f*
 - activation record on, 158*f*
- Stack pointers, 95, 98–99, 108
- Starter program, concurrency
 - analysis, 64–65
 - program structure, 61–63
- State machine, 74–77
- Static random access memory (SRAM), 20, 97, 142
- Statically allocated memory, 157–58
- Status register, 96, 211*f*, 215, 216, 218*f*, 253*f*, 259*f*, 261*f*, 263*f*, 273*f*
- Subroutines, 105–106, 140, 156–57
- Successive approximation register (SAR), 184
- Supply voltage (V^{DD}), 30, 30*f*, 31
- Switch statements, 150–52, 151*f*
- Switches, 28*f*, 29*f*, 31–32, 41
- Symbol (program), 138
- Symbol timing, 240
- Synchronous serial communications
 - protocol concepts, 249
 - clock phase and polarity, 251
 - communication, 249
 - SPI loopback test (example), 255–57
- System control, reason for, 4
- System Integration Module (SIM), 33*f*
- SysTick timer, 210
 - periodic 1 Hz interrupt (example), 212
- Target platform
 - development board, 21–22, 22*f*
 - microcontroller, 20–21, 21*f*
 - overview, 17–19
 - processor, 19–20, 20*f*
- Task deadlines, 85
- Task preemption, 84–85, 85*f*
- Task prioritization, 83–84

- Task root function, 65–66
- Tasks, concurrency, 55
 - analysis, 68–69, 68–69*f*
 - program structure, 65–68, 71*f*
- Thermocouple, 5
- Thread and Handler modes, 107
- Thumb instructions, 106–107
- Time and frequency measurement, 208
- Timer/PWM module (TPM). *See* Kinetis KL25Z timer/PWM module (TPM)
- Timers, 78, 206. *See also* Counters
 - concepts, 206
 - peripherals, 20, 21, 210
 - SysTick timer, 210–12, 210–12*f*
 - timer circuit hardware, 206, 206*f*
 - timer uses (examples)
 - periodic timer tick, 207
 - PWM signal generation, 209
 - time and frequency measurement, 208–209
 - watchdog timer, 185
- Toolchain, 134–35
- Top-of-stack, 99*f*, 100
- Transfer function, 171, 173
- Transistor, 31
- Triggering, 186
- Universal asynchronous receiver transmitter (UART), 257–62
- Unsigned extend instruction, 102
- USB drives, 14
- Vector, 111–13
- Vector table, 111, 113–14
- Volatile data objects, 127–28
- Volatile memory, 141–42
- Waiting, 82–83
- Wakeup Interrupt Controller, 20
- Watchdog timer (WDT), 207. *See also* Kinetis KL25Z COP
 - watchdog timer
- Waveform generator, 175–76
 - analysis, 299–300, 300*f*
 - design, 295–99
- Word, 97, 103–104
- Worst-case response time, 85

The Arm Education Media Story

Did you know that Arm processor design is at the heart of technology that touches 70% of the world's population - from sensors to smartphones to super computers.

Given the vast reach of Arm's computer chip and software designs, our aim at Arm Education Media is to play a leading role in addressing the electronics and computing skills gap; i.e., the disconnect between what engineering students are taught and the skills they need in today's job market.

Launched in October 2016, Arm Education Media is the culmination of several years of collaboration with thousands of educational institutions, industrial partners, students, recruiters and managers worldwide. We complement other initiatives and programs at Arm, including the Arm University Program, which provides university academics worldwide with free teaching materials and technologies.

Via our subscription-based digital content hub, we offer interactive online courses and textbooks that enable academics and students to keep up with the latest Arm technologies.

We strive to serve academia and the developer community at large with low-cost, engaging educational materials, tools and platforms.

We are Arm Education Media: Unleashing Potential

Arm Education Media Online Courses

Our online courses have been developed to help students learn about state of the art technologies from the Arm partner ecosystem. Each online course contains 10-14 modules, and each module comprises lecture slides with notes, interactive quizzes, hands-on labs and lab solutions.

The courses will give your students an understanding of Arm architecture and the principles of software and hardware system design on Arm-based platforms, skills essential for today's computer engineering workplace.

For more information, visit www.arm.com/education

Available Now:



Embedded Systems Essentials with Arm: Getting Started (on the edX platform)



Efficient Embedded Systems Design and Programming



Rapid Embedded Systems Design and Programming



Internet of Things



Graphics and Mobile Gaming



Real-Time Operating Systems Design and Programming



Introduction to System-on-Chip Design



Advanced System-on-Chip Design



Embedded Linux



Mechatronics and Robotics

Arm Education Media Books

The Arm Education books program aims to take learners from foundational knowledge and skills covered by its textbooks to expert-level mastery of Arm-based technologies through its reference books. Textbooks are suitable for classroom adoption in Electrical Engineering, Computer Engineering and related areas. Reference books are suitable for graduate students, researchers, aspiring and practising engineers.

For more information, visit www.arm.com/education

Available now, in print and ePub formats:

Embedded Systems Fundamentals with Arm
Cortex-M based Microcontrollers:
A Practical Approach
by Dr Alexander G. Dean
FRDM-KL25Z EDITION
ISBN 978-1-911531-03-6

Digital Signal Processing using Arm Cortex-M
based Microcontrollers: Theory and Practice
by Cem Ünsalan, M. Erkin Yücel, H. Deniz Gürhan
ISBN 978-191153116-6

Operating Systems Foundations with Linux
on the Raspberry Pi
by Wim Vanderbauwhede, and Jeremy Singer
ISBN 978-1-911531-21-0

System-on-Chip with Arm Cortex-M Processors
by Joseph Yiu, Distinguished Arm Engineer
ISBN 978-1-911531-19-7

Arm Helium Technology
M-Profile Vector Extension (MVE) for Arm Cortex-M
Processors
by Jon Marsh
ISBN: 978-1-911531-23-4



